Computational Intelligence

WILEY

Computational Intelligence

ORIGINAL ARTICLE

Feature-Driven Deep Architecture for High-Precision Recognition of Complex Handwriting

Arvind Mewada[1] | Mohd. Aquib Ansari[2] | Shahnawaz Ahmad[1]

[1]School of Computer Science Engineering and Technology, Bennett University, Greater Noida, Uttar Pradesh, India | [2]SCSE, Galgotias University, Greater Noida, Uttar Pradesh, India

Correspondence: Shahnawaz Ahmad (shahnawaz98976@gmail.com)

Received: 25 August 2025 | **Revised:** 30 March 2026 | **Accepted:** 7 May 2026

Keywords: character recognition | deep convolutional neural network | explainable artificial intelligence | handwritten documents | hybrid optimization

ABSTRACT

Handwritten character recognition (HCR) plays a crucial role in digitizing documents, especially in developing countries where much information remains in hard copy. While automation improves accessibility and preservation, challenges such as variations in handwriting style, size, and image quality persist. Traditional models struggle with these variations, and while deep learning models are accurate, they often lack efficiency and generalization. To address this, we propose an adaptive HCR framework combining advanced preprocessing, feature extraction, and bio-inspired optimization. Our approach uses a novel HOG-Saliency-Structural Feature Map (HOG-SSFMap) that enhances feature learning by integrating Histogram of Oriented Gradients, saliency, and structural features. For classification, we utilize a PSILO-DNet model based on DenseNet121 optimized by Paniscus Social Interaction Learning Optimization (PSILO), inspired by primate group behavior. This model ensures stable and accurate convergence. Experiments on benchmark datasets show promising results, achieving 96.01% accuracy on Devanagari and 95% on PHDIndic_11, proving its scalability and effectiveness.

1 | Introduction

For hundreds of years, people have been reliant on handwritten documents as essential information archives, including family histories, land records, birth certificates, religious writings, and educational materials. In most developing countries, a lot of such records are either handwritten or printed and have not been digitized. This is a colossal disparity in the current modern and technology-driven society. The digitization of these documents is the key to making them more available and protecting them [1, 2]. HCR systems are suggested to convert the images of handwritten documents into text through computer vision steps, which is considered to make the images machine-readable and encounter many difficulties. These are inherent differences in the variability of handwriting styles, size, and orientation and differences across writers over time [3]. These mismatches have made HCR one of the most complex paintings in document image processing. The systems are challenged by noisy scanned images, illegible handwriting, overlapping characters, as well as a number of other factors that reduce recognition precision.

One of the most sophisticated frameworks, which transforms the print and handwritten text to electronic or machine-readable formats, is Optical Character Recognition (OCR) [4, 5]. OCR has found a wide application in speech processing, medical records and biometric authentication systems because of its versatility. OCR is most effective with printed text; recognition of handwritten text is much harder because of the variation of size, style and position of the letters in a word. Some methods in the machine learning fields have been created to enhance the accuracy of the handwriting recognition process, among them being the Random Forest, Support Vector Machines (SVM), and Artificial

© 2026 Wiley Periodicals LLC.

Neural Networks (ANN) [6]. Models using labeled training data have the ability to learn a character classifier but have problems with background noise, complex script images, and inter- and intra-writer variability. As a result of the introduction of deep learning, there have been major advances in HCR, such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and alternative hybrid architectures. CNNs are good at extracting features in images, especially because they have the capability of extracting and utilizing different distinguishing features or attributes of an image without additional effort to identify features. Deep learning (DL) models [7] have emerged as a powerful and flexible solution that results in higher performance on various handwriting datasets.

However, deep learning of HCR is among the spheres that have not been completely refined. Irrespective of the progress of these models, they can do better than humans; they are grappling with HCR. Handwriting is most of the time not legible, misshapen or poor, and hence difficult to recognize. Even though there are complicated features in the model of DL, it still struggles with the issue of space relationships in high-dimensional data. To address these issues, hybrid models have been created in terms of various methods to improve accuracy. Nonetheless, it takes more effort to handle when it comes to changes in handwriting styles, sizes, and quality. The variations require that the deep learning models be flexible. Other areas of application of deep learning include the detection of fatigue in online learning systems and education monitoring. Regardless of these developments, however, HCR is a complex field that needs additional development since, as yet, other multi-level and flexible models are required to render optimal precision and consistency in practice.

It was evident in the ECA-Net model [8] that generalization was notably low in processing found in the model, which led to lower recognition accuracy. Conversely, the GSO-NN model of [9] made attempts to take account of actual-time datasets, though it was hampered by the size of the sample, making the accuracy of such a sparse dataset erroneous in estimating the time-varying sample dispersion. Due to stylistic sentence casing, the Nearest Quad Tree (NQT) classifier [10] could not capture the underlying features of handwriting, which hindered recognition performance. Extraction of features in CNN-based models is inadequate, leading to lowered recognition rates alongside high memory consumption, which resulted in poor model performance [11]. Similarly, the CapsNet [8] model did not perform well on recognizing identity information in handwritten documents and suffered from low-level computer vision tasks, decreasing recognition performance. All these models emphasize the need to have more precise, effective and generalized models of handling various samples of handwriting in real-life situations.

In order to overcome these shortcomings, the originality of this piece has been made very clear. In comparison with the current hybrid CNN-based models, the proposed one presents a tri-modal feature fusion model with metaheuristic optimization, which provides the possibility to learn features complementary to each other and higher convergence stability. Particularly, four aspects are the major novelty. To begin with, a tri-modal feature fusion system (HOG 3S) is proposed, with HOG being used to encode information on edges and shapes, saliency maps being used to encode regions of attention, and structural maps being used to encode local texture patterns, which makes a complete representation of handwritten data. Second, unlike traditional approaches in which merging features is done at the classifier level, the given approach fuses features at the feature level, before CNN processing through forming a structured multi-channel input, hence enhancing the inductive learning of the network. Third, the hybrid optimization approach that combines PSILO with DenseNet121 is suggested, in which PSILO will globally search on final layer weights, minimize the local minima problem and become more resilient to contaminated handwriting samples. Lastly, it has a contribution in the synergetic combination of preprocessing, feature fusion, deep learning, and bio-inspired optimization, and not isolated gains, which will lead to a more robust and precise recognition structure.

In order to address these issues, we propose a unique solution, establishing a new, flexible and strong structure of HCR. Our method was made by incorporating higher-level preprocessing to improve image clarity, hybrid feature-expansion to get pertinent visual features, and the classification of a DL model that utilizes bio-inspired intelligence to improve the performance accuracy. There was a great improvement in recognition accuracy in various handwriting styles that were tested against Devanagari [12] and PHDIndic_11 datasets [13]. The list of the key contributions of this article has the following entries:

- To improve recognition performance, we outlined a preprocessing pipeline for noise reduction and image quality enhancement.
- We proposed a new feature extraction mechanism, the HOG-Saliency-Structural Feature Map (HOG-SSFMap), which integrates the Histogram of Oriented Gradients (HOG), Hybrid Saliency Map, and Modified Structural Map.
- We added the PSILO-DNet model, which merges transfer learning with DenseNet121 to offer rich deep feature representations.
- We applied the PSILO algorithm with hybrid intelligence from chimpanzee and spider monkey behavior to enhance classification, guiding stable convergence without local minima.
- We showed improved recognition performance by conducting extensive evaluations on the Devanagari and PHDIndic_11 datasets, realizing significant accuracy and efficiency improvements.

The other sections of this paper are structured as follows: Section 2 entails a comprehensive review of the literature on HCR. Section 3 clearly describes the methodology, the propositions containing mathematical equations. Section 4 includes a comparison of the experimental findings of the developed recognition model with the other literature models. Lastly, Section 5 provides a conclusion of the article and talks about the possible future research directions.

2 | Literature Review

The section is a review of literature on the topic of Handwritten Character Recognition (HCR) and Handwritten Text Recognition

(HTR), including the methods they use, their performance results, and limitations. They involve the standard deep-learning models as well as the latest progress (2025–2026) in transformer-based models and effective recognition systems. One model is presented by Aditi Moudgil et al. [14] called Capsule Network (CapsNet). This model is used to recognize the regional scripts like Gurmukhi, Devanagari and Sanskrit. The model is quite effective in representing spatial relationships between characters, and hence, makes correct recognition possible. Nevertheless, parameters overlapping in the process of manuscript digitization had led to the loss of information. Tang Minli et al. [15] proposed a lightweight Artificial Neural Network (ANN) that is more effective in feature extraction and enhancing the accuracy of the detection process, but the multimodal features, such as speech, cannot be processed by the device, which constrains its use in translation systems. Anfal Bin Durayhim et al. [11] exploited a Convolutional Neural Network (CNN), which was able to achieve high detection rates and low computational costs, but failed to provide good performance in more complicated handwriting cases because of poor feature representation. On the same note, Saleh Alwagdani and Emad Sami Jaha [16] tested CNN models on Hijja and AHCD datasets with high accuracy but with deliberate prediction errors. Moudgil et al. [17] also suggested a CapsNet-LSTM hybrid structure to enhance understanding and recognition performance, but its application is still weak. Ashlin Deepa et al. [10] created a Nearest Quad-tree (NQT) classifier on Tamil, Telangana and Devanagari datasets, which could not perform generalization across styles of handwriting. A GSO-NN model proposed by Khalid Nahar et al. [9] had a high accuracy with minimal error, but due to the lack of training data, the model is limited in its ability. Qing-Dao-Er-Ji Ren et al. [8] suggested ECA-Net, which has good performance on large datasets but with high computational complexity and low-generalization. Wei Chen et al. [18] proposed Fine-Grained Automatic Augmentation (FgAA), a data augmentation method, where the stroke-level transformations are modeled with the help of Bézier curves and the augmentation strategies are optimized using Bayesian optimization. Although it is better in performance on multilingual datasets, it needs large datasets and parameter tuning. Recent works have been moving towards transformer-based architectures, efficient decoding and applicability to real-world situations. Yuting Li et al. [19] designed a vision transformer architecture (HTR-VT) that has been shown to perform highly with little labeled data. They combine CNN-based feature extraction and Sharpness-Aware Minimisation (SAM) to enhance generalization and a span masking strategy, which is used as a regulariser. The model will perform state-of-the-art on datasets like IAM and READ2016 without having to pretrain it extensively. The Meta Document Attention Network (Meta-DAN) is a new autoregressive decoding scheme of page-level handwritten text recognition proposed by Denis Coquenet [20]. The model also uses windowed queries and multi-token predictions to minimize the time taken to infer the query and enhance contextual modeling. The method is able to deliver state-of-the-art performance on a variety of datasets and offers a trade-off between speed and accuracy. Meyer et al. [21] suggested that transformer-based models be changed with an n-gram injection (NGI) technique to deal with shifts in language distribution. The approach is applicable across languages and cross-domains with an optional interpretation of the external language models, which resulted in enhanced recognition accuracy without the need to retrain the model, thus making it multilingual. According to a recent survey by Garrido-Muñoz et al. [22], HTR has evolved to incorporate deep learning and transformer-based techniques as opposed to the traditional handcrafted techniques. Among the main challenges identified in the study are: no standardized evaluation frameworks, weak cross-dataset generalization, reliance on synthetic data and weak reproducibility. It further highlights new trends in the form of Vision-Language Models (VLMs) and Self-Supervised Learning (SSL) to solve data scarcity. In terms of application, Martuscelli et al. [23] tested HTR on an industrial scale and showed that although the HTR systems were able to shorten the processing time by more than 70%, the recognition accuracy is not steady and needs verification by humans. This demonstrates the disconnect between theory and practice. Table 1 is a comparative study of currently used methods of HCR/HTR, including their strengths and weaknesses.

Despite significant progress in handwritten character and text recognition, several challenges continue to limit the effectiveness and practical deployment of existing methods. A major challenge is the lack of generalization ability of models to other datasets and languages, with a lot of methods being trained and tested on particular datasets and with limited ability to be generalized to handwriting styles and scripts of various types. Also, there is a high reliance on large-scale or synthetic data, which cannot necessarily capture the variability in the real world and can produce biased results. Most modern deep learning and transformer-based models, despite their power, are resource-intensive and can be deployed in either high-resource or low-resource settings with high computational complexity. Moreover, autoregressive architectures cannot be used in time-sensitive tasks because the decoding is slow and sequential. The other major issue is that models are sensitive to changes in handwriting, noise, and document quality, which may deteriorate the recognition of documents. Lastly, although many systems yield positive results in the research, their implementation in practice has limitations because of varying levels of accuracy, which require human intervention and make complete automation less possible.

3 | System Model of Handwritten Character Recognition

Figure 1 presents a schematic overview of the proposed HCR system, which includes four key stages: preprocessing, feature extraction, training, and output generation. In the preprocessing stage, the quality of the input handwritten text image is improved by removing noise and enhancing visual clarity. The system sends the refined image to the next functional stage to identify important and distinctive characteristics. After receiving the extracted features, the model studies diverse character types through the training phase. The system generates exact recognition outputs through the trained model during the output phase.

3.1 | Proposed Methodology

With the intention of achieving a salient HCR system, we propose a PSILO optimized DenseNet architecture that includes fine-tuned feature evaluation of encoded gradient, structural,

TABLE 1 | Comparison of handwritten character recognition methods: Strengths and Limitations.

Authors	Model used	Strengths	Limitations
Aditi Moudgil et al. [14]	CapsNet	Captures spatial relationships in characters	Overlapping parameters cause data loss
Tang Minli et al. [15]	Lightweight ANN	Efficient feature extraction; improved accuracy	Limited multimodal capability
Anfal Bin Durayhim et al. [11]	CNN	High accuracy; low computational cost	Weak feature representation
Alwagdani & Jaha [16]	CNN	High recognition accuracy	Prediction errors
Moudgil et al. [17]	CapsNet-LSTM	Improved interpretability	Limited practical applicability
Ashlin Deepa R. N et al. [10]	NQT Classifier	Works on regional datasets	Poor generalization
Khalid M. O. Nahar et al. [9]	GSO-NN	High accuracy; minimal errors	Limited training data
Qing-Dao-Er-Ji Ren et al. [8]	ECA-Net	Effective on large datasets	High computational cost
Wei Chen et al. [18]	FgAA	Improved augmentation and accuracy	Requires large datasets
Yuting Li et al. [19]	ViT + CNN + SAM	Data-efficient transformer; strong performance	Computational complexity
Denis Coquenet [20]	Meta-DAN	Faster decoding; better context modeling	Still computationally demanding
Meyer et al. [21]	NGI (Transformer + *n*-gram)	Handles language shift dynamically	Depends on external language models
Garrido-Muñoz et al. [22]	Survey	Comprehensive analysis of HTR methods	Highlights the lack of standardization
Martuscelli et al. [23]	Industrial HTR system	Real-world validation; improved efficiency	Low accuracy; needs human validation

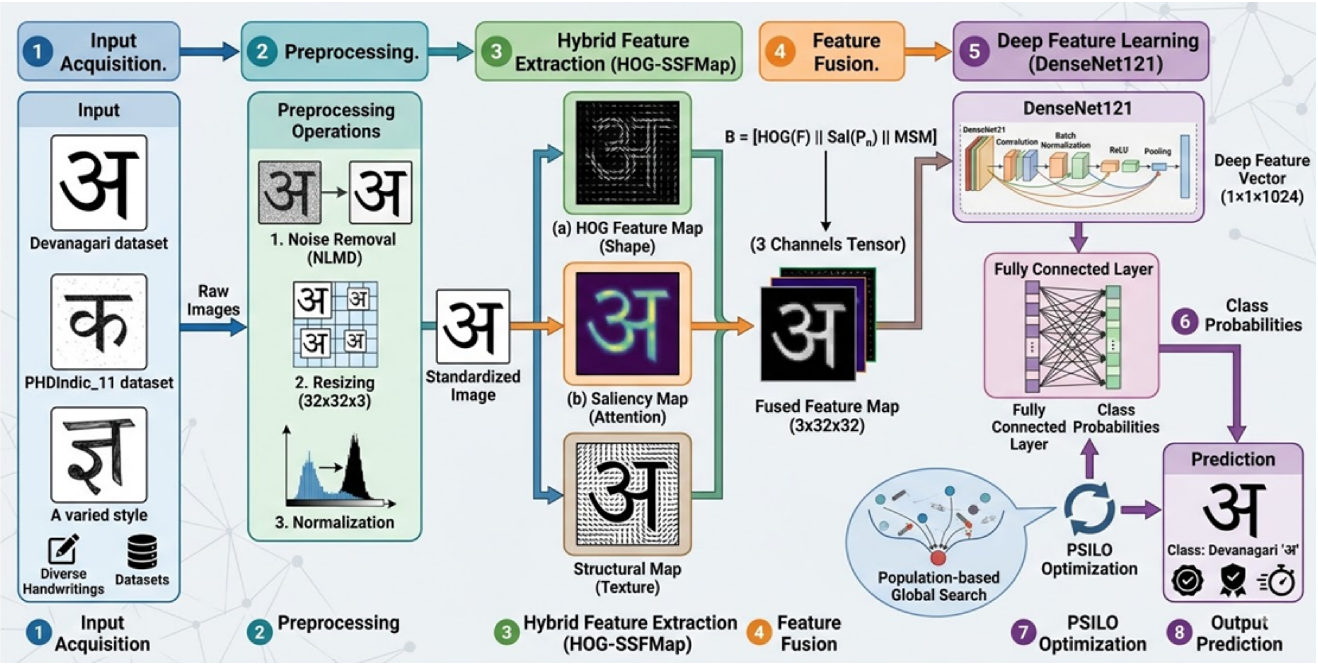


FIGURE 1 | Framework representation of the proposed HOG-SSFMap-PSILO-HDNet architecture.

and saliency feature maps. To achieve this task, the image to be input is first pre-processed to improve the quality by removing noise and irrelevant artifacts. The processed image is then sent to the feature extraction stage, where useful as well as important features are constructed. A custom mechanism for extraction, HOG-Saliency-Structural Featural Map (HOG-SSFMap), is developed here to capture distinguishing features by employing Histogram of Oriented Gradients, Hybrid Saliency Map, and Modified Structural Map handcrafted methods. The fractured feature maps are combined to form a single set fed to the PSILO-DNet model, which uses the PSILO-enabled DenseNet121 architecture to provide better feature representation. The PSILO

algorithm is implemented with enhanced hybrid behavior from chimp and spider monkeys to guarantee thorough convergence without getting entrapped in local optima. This combined strategy provides excellent results for HCR. The architecture of HOG-SSFMap-PSILO-DNet is presented in Figure 1. A detailed discussion of the technical aspects of this proposed methodology is presented as follows:

3.2 | Handwritten Image Input

The research makes use of source data from two different datasets: PHDIndic_11 [24] and Devanagari [25] that incorporate information regarding 11 Indic scripts and the Devanagari script. The datasets include numerous handwritten character examples from various Indian languages, allowing the model to detect numerous character types. Mathematically, the input data appears as:

$$H = \left\{H_1, H_2, \ldots H_n, \ldots . H_r\right\}_{b\times c\times d} \tag{1}$$

where H indicates the dataset with r number of inputs, and H_n is the nth input. Further, b, c, d is the height, width, and channel of the input.

3.3 | Pre-Processing of Input Image

Image analysis Preprocessing is an important process as it improves image quality and eliminates noise to improve the accuracy of the further stages in the image processing. This piece of work performs denoising, normalization, and resizing of the input image with the aim of enhancing the extraction of robust features and the training of the model. At the beginning, the Non-Local Means Denoising (NLMD) [26] is used to minimize noise and keep small structural conditions. Contrary to local filters, NLMD calculates the pixel by averaging similar patches in the entire image, by weighted averaging, which makes it very successful in handwritten data that has uneven noise distributions. $I'(x, y)$is the denoised pixel value, which is calculated mathematically as:

$$I'(x, y) = \frac{\sum_{i,j} w(x, y, i, j) \cdot I(i, j)}{\sum_{i,j} w(x, y, i, j)} \tag{2}$$

where the weight function is defined as follows:

$$w(x, y, i, j) = exp\left(-\frac{\|I(x, y) - I(i, j)\|^2}{h^2}\right) \tag{3}$$

Here, I(x,y) and I(i,j) represent pixel intensities, and h controls the degree of smoothing. This formulation ensures that pixels with similar intensity patterns contribute more significantly, thereby preserving edges and textures.

After the denoising step, the image is scaled to a constant size of $32 \times 32 \times 3$. Such resizing guarantees compatibility with the DenseNet-based framework, which requires inputs to be of uniform size to be processed successfully. It also makes training and inference faster by lowering computational complexity. Meanwhile, a size that has been selected has an adequate balance between the resolution and efficiency, thus it is highly applicable to large-scale datasets without substantial loss of valuable visual data. In order to test the resizing effects, another experiment was carried out with a 64×64 resolution, which only led to a small improvement of +0.3% in accuracy and significantly increased the computing expense. This proves that the 32×32 resolution of the selected framework is the best. Even though resizing can cause a small loss of information, the latter can be partially compensated by the next extraction phase (HOG–Saliency–Structural maps), which is needed to preserve the important edge, texture and spatial details. Lastly, the processed image is normalized and presented as a k × l × m-sized $32 \times 32 \times 3$ tensor. This orderly input increases the stability of the model, decreases the variability that is caused by noise and improves the overall recognition.

3.4 | Generation of HOG–Saliency–Structural Feature Map

This article suggests a HOG-Saliency-Structural Feature Map (HOG-SSFMap), which offers a concise and selective portrayal of handwritten images. The feature map is made up of three feature channels that are complementary, and they are HOG (edge/shape), Saliency (attention), and Structural Map (texture). Each channel represents different visual information and is eventually combined into one representation. Their choice is associated with the complementary properties of these feature maps to represent edge, attention, and texture data of (written) data, resulting in increased resilience to changes in writing style, noise, and distortions.

a. ***Histogram of Oriented Gradients (HOG) Feature Map***

The HOG represents local edge and gradient structures that are necessary to describe character shapes. The gradients have been calculated as follows:

$$G_x = I(x + 1, y) - I(x - 1, y) \tag{4}$$

$$G_y = I(x, y + 1) - I(x, y - 1) \tag{5}$$

$$G = \sqrt{G_x^2 + G_y^2} \tag{6}$$

$$\theta = tan^{-1}\left(\frac{G_y}{G_x}\right) \tag{7}$$

The image is decomposed into cells and the gradient orientations are added up to histograms. The following histograms are normalized to enhance strength:

$$D_b = \frac{H_b}{\left|H_b\right|_2 + \epsilon} \tag{8}$$

This is implemented by first resizing the input picture to 32×32 pixels and changing the picture to grayscale. The computation of gradients is done with the Sobel filters, and then the histogram is computed over the local cells, and the block normalization is done through the L2-Hys technique. The resulting feature map of HOG is a 32×32 map. The computational process is elaborate, as shown in Algorithm 1.

ALGORITHM 1 | HOG Feature Map.

1. **Input:** Image I of size 32×32
2. **Output:** HOG feature map F of size 32×32
3. **Function ComputeGradient(I):**
4. Convert I to grayscale -> I_gray
5. G_x <- I_gray(x+1, y) - I_gray(x − 1, y)
6. G_y <- I_gray(x, y+1) - I_gray(x, y − 1)
7. Magnitude G(x, y) <- sqrt(G_x^2 + G_y^2)
8. Orientation θ(x, y) <- atan2(G_y, G_x)
9. Return G, θ
10. **Function ComputeCellHistograms(G, θ, cell_size):**
11. Divide the image into non-overlapping cells of size cell_size × cell_size
12. For each cell:
13. Initialise histogram H_b with B bins
14. **For each pixel (x, y) in cell:**
15. bin <- quantize(θ(x, y))
16. H_b[bin] <- H_b[bin] + G(x, y)
17. Return all cell histograms
18. Function NormalizeBlocks(cell_histograms, block_size):
19. **For each block (group of neighboring cells):**
20. H_block <- concatenate histograms of cells in block
21. D_b <- H_block/(||H_block||_2 + ε)
22. Return all D_b
23. **Function ComputeHOG(I):**
24. G, θ <- ComputeGradient(I)
25. cell_histograms <- ComputeCellHistograms(G, θ, cell_size=8)
26. D_b <- NormalizeBlocks(cell_histograms, block_size=2×2)
27. F <- concatenate all D_b
28. **Return F**

b. ***Hybrid Saliency Map (FASA-Based)***

The saliency map is a model that uses the FASA model to create a saliency map based on spatial distribution and global contrast. The saliency probability is computed with a formulation based on a Gaussian, and global contrast enhances foreground-background discrimination. Pn Saliency probability of each pixel Pn is computed as follows:

$$p(P_n) = \mathcal{N}\left(P_n|\mu, \Sigma\right) \tag{9}$$

where $\mu$and Σrepresent the mean and covariance of the feature distribution, respectively.

The Global contrast is defined as follows:

$$N_{con}(P_n) = \sum w(J_{an}, J_{hn}) \tag{10}$$

where $w(\cdot)$denotes the weight between quantized color features.

Finally, the saliency map is computed by combining probability and contrast information:

$$Sal(P_n) = \frac{\sum w \cdot p(P_n) \cdot N_{con}(P_n)}{\sum w} \tag{11}$$

The algorithm is carried out by calculating the spatial center and variance of the pixels and quantizing the colors to decrease the number of features. This is followed by an estimation of saliency probability that is estimated using a Gaussian model, which is further refined on global contrast measures. The saliency map is the resulting 32×32 pixel saliency map that shows the most salient regions in the image. The entire algorithm is found in Algorithm 2.

ALGORITHM 2 | FASA-Based Hybrid Saliency Map Computation.

1. **Input:** Image F taken from HOG output of size 32×32
2. **Output:** FASA saliency map Sal(P_n) of size 32×32

3. **Function BinaryStructure(I, x, y):**
4. BS <- 0
5. Define N(x, y) as the 3x3 window around (x, y)
6. I_cen <- I(x, y) // Intensity of center pixel
7. For each pixel g in N(x, y):
8. If I_g >= I_cen:
9. BS <- BS + 1
10. Return BS

11. **Function TernaryStructure(I, x, y):**
12. TS <- 0
13. Define N(x, y) as the 3x3 window around (x, y)
14. I_cen <- I(x, y) // Intensity of center pixel
15. For each pixel g in N(x, y):
16. If I_g > I_cen:
17. TS <- TS + 1
18. Else if I_g < I_cen:
19. TS <- TS - 1
20. Return TS

21. **Function ComputeSpatialCenterAndVariance(I):**
22. For each pixel P_n, compute S_x(P_n) <- spatial center and U_x(P_n) <- variance
23. Use the formulas (6) and (7) to calculate using color and positional vectors

24. **Function QuantizeColors(I):**
25. J_an <- Quantize colors in image I to obtain quantized color vector

26. **Function ComputeProbabilityOfSaliency(P_n, S_x, U_x):**
27. p(P_n) <- Compute probability using the multivariate Gaussian function (8)
28. G_n <- Compute Gaussian matrix
29. Apply Gaussian formula to compute p(P_n)

30. **Function ComputeGlobalContrast(I, J_an):**
31. N_con(P_n) <- Compute global contrast using J_an and formula (10)

32. **Function ComputeFASA(I):**
33. Initialize Saliency Map Sal(P_n) <- empty matrix of size 32x32
34. For each pixel P_n in I:
35. BS_value <- BinaryStructure(I, x, y)
36. TS_value <- TernaryStructure(I, x, y)
37. p(P_n) <- ComputeProbabilityOfSaliency(P_n, S_x, U_x)
38. N_con(P_n) <- ComputeGlobalContrast(I, J_an)
39. Sal(P_n) <- Σ(w(J_an, J_hn) * p(P_n) * N_con(P_n))/Σ(w(J_an, J_hn))
40. Return Sal(P_n)

c. ***Modified Structural Map (MSM)***

The Modified Structural Map is a variant of this type of map that represents local texture and pixel-level structural relationships

with binary and ternary comparisons. Binary Structure (BS) 3×3, Ternary Structure (TS) 3×3 are calculated around every pixel to obtain fine-grained structural details. The Binary Structure is stated to be:

$$BS(x, y) = \sum I\left(I_g \geq I_{\text{cen}}\right) \tag{12}$$

where I_{cen}represents the intensity of the center pixel and $I_g$denotes the neighboring pixels within the 3×3 window.

The Ternary Structure is computed as follows:

$$TS(x, y) = \sum \left[I\left(I_g > I_{\text{cen}}\right) - I\left(I_g < I_{\text{cen}}\right)\right] \tag{13}$$

These two representations are effective in representing local variation in intensity and structural correlation. The implementation uses local neighborhood analysis to calculate BS and TS values, and subsequently fuses them with the help of a given strategy. In this work, the maximum fusion strategy is taken to preserve the most dominant structural responses:

$$MSM(x, y) = \max \backslash big(BS(x, y), TS(x, y) \backslash big) \tag{14}$$

The resulting structure is a 32×32-sized structural feature map that is useful to capture texture and structural features. The steps are explained in Algorithm 3.

d. **Feature Fusion (Final Representation)**

The three extracted feature maps are integrated to form a unified 3-channel representation, expressed as follows:

$$MSM(x, y) = \min \backslash big(BS(x, y), TS(x, y) \backslash big) \tag{15}$$

In which HOG(F) is the gradient-based structural features, Sal(Pn) is the saliency-enhanced probability map, and MSM is the structural feature map. This integration is practical in that it integrates the complementary information in shape, attention, and texture domains into one representation. Table 2 provides a stage-by-stage visual representation of the entire process of changing the input image into the final fused representation. Consequently, it minimizes noise and redundancy of raw images without losing discriminating features. Moreover, this combined representation makes convergence of the CNN model easier and makes the model better at generalization, as well as more resistant to differences in handwriting.

ALGORITHM 3 | Modified Structural Map.

1. **Algorithm:** ComputeModifiedStructuralMap
2. **Input:** Grayscale image of size 32x32
3. **Output:** Modified Structural Map (MSM) of the same size
4. Function BinaryStructure(I, x, y):
5. **Input:** Image I, coordinates (x, y)
6. **Output:** Binary Structure BS(x, y)
8. BS(x, y) <- 0
9. Define neighborhood N(x, y) as the set of pixels around (x, y) within a 3x3 window
10. I_cen <- I(x, y) // Center pixel intensity
11. **For each pixel g in N(x, y):**
12. If I_g >= I_cen:
13. BS(x, y) <- BS(x, y) + 1
14. Return BS(x, y)
15. **Function TernaryStructure(I, x, y):**
16. Input: Image I, coordinates (x, y)
17. Output: Ternary Structure TS(x, y)
19. TS(x, y) <- 0
20. Define neighborhood N(x, y) as the set of pixels around (x, y) within a 3x3 window
21. I_cen <- I(x, y) // Center pixel intensity
22. For each pixel g in N(x, y):
23. If I_g > I_cen:
24. TS(x, y) <- TS(x, y) + 1
25. Else if I_g < I_cen:
26. TS(x, y) <- TS(x, y) - 1
27. Return TS(x, y)
28. **Function ComputeModifiedStructuralMap(I):**
29. **Input:** Image I
30. **Output:** Modified Structural Map MSM
32. Initialize MSM as an empty matrix of the same size as I (160×160)
34. **For each pixel (x, y) in the image:**
35. BS_value <- BinaryStructure(I, x, y)
36. TS_value <- TernaryStructure(I, x, y)
38. // Fusion strategy: Maximum, Minimum, or Mean
39. // For Maximum fusion:
40. MSM(x, y) <- max(BS_value, TS_value)
42. **// For Mean fusion:**
43. // MSM(x, y) <- (BS_value + TS_value)/2
45. // For Minimum fusion:
46. // MSM(x, y) <- min(BS_value, TS_value)
48. Return MSM.
49. **Main Process:**
50. a. Load the grayscale image I of size 32×32
51. b. Call ComputeModifiedStructuralMap(I) -> MSM
52. c. Return the computed

3.5 | Optimized DenseNet Network

It is a complex task, and suggestive architecture: PSILO-assisted DenseNet121, which is proposed to accomplish Handwritten Character Recognition (HCR) [7, 27]. Being a highly connected convolutional neural network, DenseNet121 is positioned to tackle the following few important issues, namely the vanishing gradient problem, redundancy of features, and over-parameterization. In this architecture, every layer takes inputs as outputs of all the previous layers, and the output is sent to all the following layers, so that features can be easily reused and gradient flow is enhanced. The HOG-SSFMap feature maps are input into the DenseNet121 as the backbone through transfer learning. This allows the model to use pre-trained weights, which have usually been trained on large-scale datasets, like ImageNet, and make them fit the HCR task. The input image in the form of a $32 \times 32 \times 3$ object is manipulated by a first convolutional block, which includes ReLU activation, batch normalization, max pooling, and dropout layers, resulting in an output of $1 \times 1 \times 1024$. As Figure 2 demonstrates, these operations improve the stability of the training and reduce the overfitting. DenseNet121 is a network that is made up of four dense blocks with three transition

TABLE 2 | Image outcomes of HOG-saliency-structural feature map.

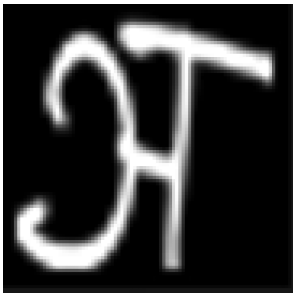
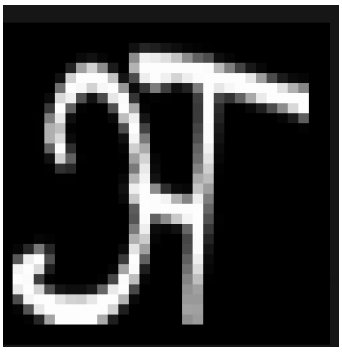
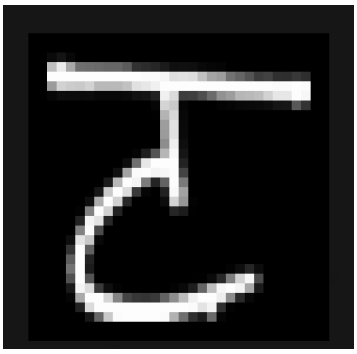
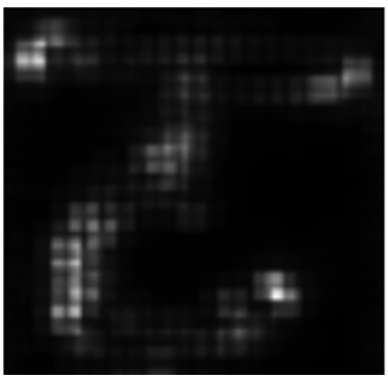
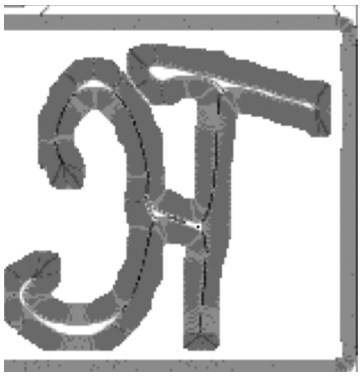
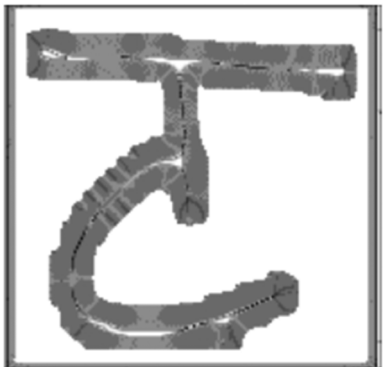
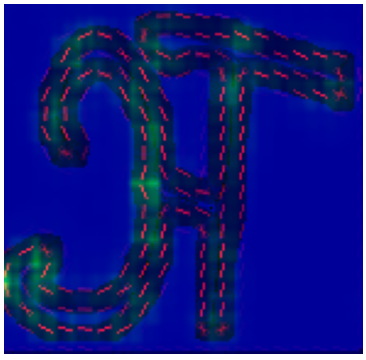
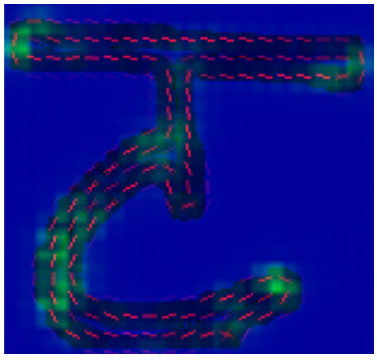
Methods/outcomes	Sample 1	Sample 2
Input		
Preprocessing		
HOG Feature Map		
Hybrid Saliency Map		

TABLE 2 | (Continued)

Methods/outcomes	Sample 1	Sample 2
Modified Structural Map		
Overall Feature Fusion (HOG-SSFMap) $B = [HOG(F) \,\|\|\, Sal(P_n) \,\|\, MSM]$		

layers. Both transition layers utilize 1×1 convolutions and 2×2 average pooling to downsize spatial dimensions and scale of computation. Following the last dense block, a global average pooling layer is used to reduce the feature representation to a $1 \times 1 \times 1024$ representation, which is flattened and then fed to a fully connected feedforward neural network (FFNN) to do the classification. As an additional performance optimization, the PSILO (Paniscus Social Interaction Learning Optimization) algorithm is used on the final fully connected layer prior to the softmax output. PSILO is a population-based metaheuristic, unlike the traditional gradient-based optimizers (i.e., SGD or Adam), as it searches the entire space of possible weight configurations to find the best ones. This strategy reduces the cross-entropy loss and eliminates local minima, and enhances convergence. The optimization process is decoupled effectively by using PSILO during the last stage, and makes the optimization susceptible to noisy gradients and bad initializations. As depicted in Figure 2, this integration results in improved classification accuracy, better decision boundary learning, and efficient convergence without significantly increasing computational cost.

3.6 | Paniscus Social Interaction Learning Optimization (PSILO)

A mandatory step towards improving the HCR is to implement the PSILO algorithm [28] in the DenseNet121 model. This algorithm implements attributes like social organization and interaction of the spider monkey and the fission-fusion strategy of the chimps. Adaptability in complex environments represents pattern navigation, which simulates the illustration of various handwriting recognition challenges. Reliant on communication and collaboration, social structures give rise to algorithms with improved information sharing, precision, and accuracy, and academically hyphenated recognition. On the other hand, chimps are remarkable learners and problem solvers. Such hybridization prevents the performance degradation of the research model, succeeding in the optimal recognized outcomes.

3.6.1 | Inspiration

Social structure and interaction attributes stem from Simia Paniscus, which is of the family Atelidae [25]. The Paniscus generally lives in a group, where learning is performed to attain efficient solutions. In addition, the characters of the fission-fusion strategy and the social organization are inspired by the *Pan troglodytes* [23] from the family Hominoidea. Combining these inspirations, the performance and adaptability of PSILO-DNet in HCR are increased, and optimal outcomes are attained in the research.

3.6.2 | Initialization

The solutions of the PSILO algorithm are initialized, which tune the hyperparameters of the ResDNet model. The initialized solutions are mathematically formulated as,

$$A_{q,t} = A(0,1)\left(Amin_{max}\right)_{min} \tag{16}$$

where $q$indicates the q^{th}solution, t is the tth direction, $Amin_{max}$ indicates the upper and lower boundaries of the search space,

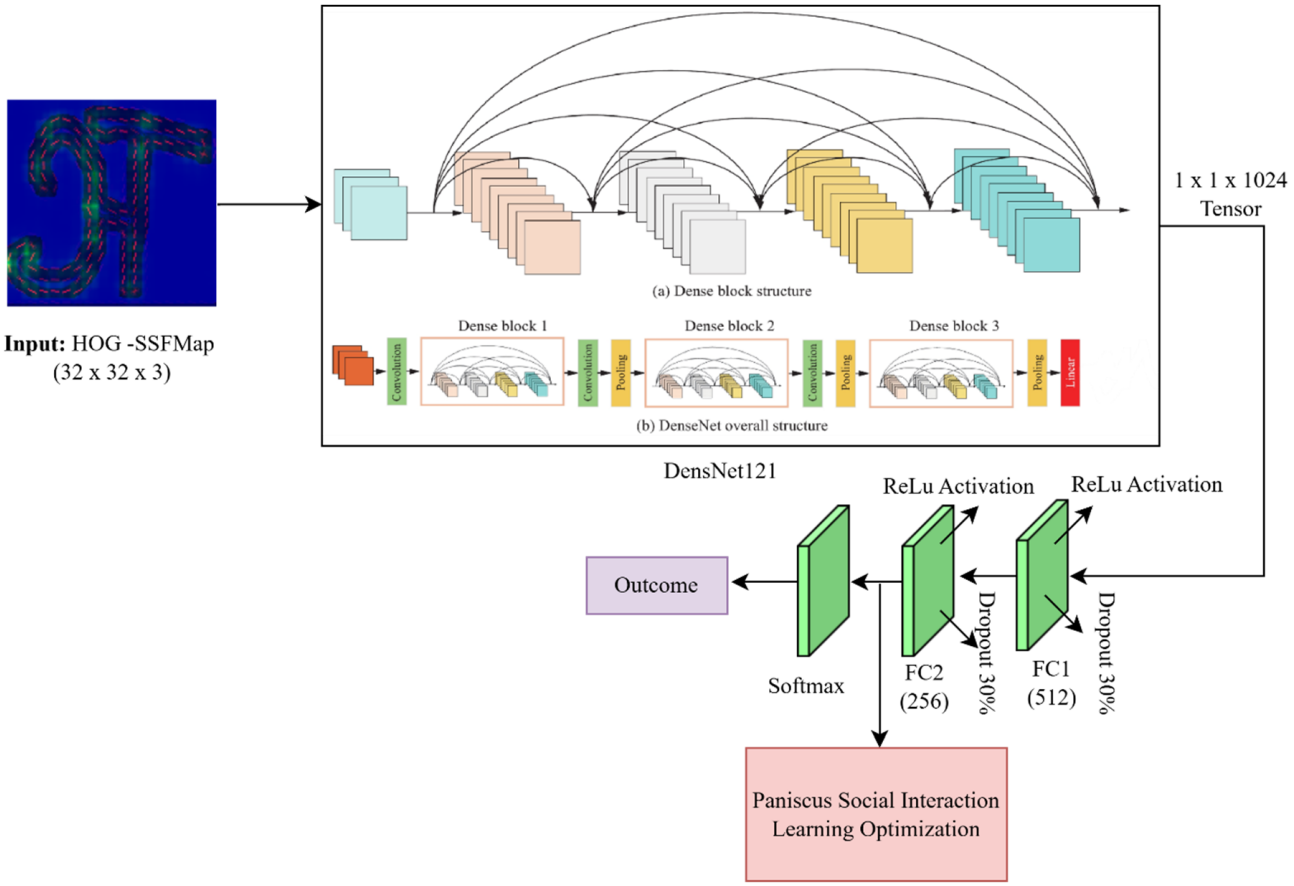


FIGURE 2 | Architectural depiction of PSILO-DNet model.

$M(0,1)$indicates the uniformly distributed random number that ranges from $(0,1)$ and the total number of populations in the solution is z.

3.6.3 | Fitness Evaluation

The fitness evaluated in the algorithm remains maximum, as it declares the efficient outcomes attained in the research model. The fitness of the algorithm is declared as,

$$W\left(A_{q,t}\right) = max\left[\chi\left(accuracy\left(A_{q,t}\right)\right) + (1-\chi)\left(precision\left(A_{q,t}\right)\right)\right] \tag{17}$$

where χ is the constant coefficient, $W\left(A_{q,t}\right)$ indicates the fitness of the solution $A_{q,t}$.

3.6.4 | Solution Update

The solution update of the PSILO algorithm is based on each criterion combined in the optimizations. The combined characteristics are explored in the solution update, where the best solutions are declared to tune the hyperparameters significantly.

3.6.4.1 | Phase 1: Local Leader updation Phase. As the solutions declare the local leader in general, the updation of their position is based on the entire group's updation. The communication between the leader and the group's solutions declares the updation of a leader at iterations. The group solutions of the population can be classified into driver solutions $\left(A_{dvr}\right)$, barrier solutions $\left(A_{bar}\right)$, chaser solutions A_{chsr}, and attacker solutions $\left(A_{atkr}\right)$. Furthermore, the best position of all other group solutions declares the updation of the leader, which is mathematically formulated as,

$$A_{q,t}^{K+1} = A_{q,t} + M(0,1) \times \left(A_{ll,t} - A_{q,t}\right) + M(-1,1) \times \left(A_{rand,t} - A_{q,t}\right) \tag{18}$$

where $A_{ll,t}$is the local leader, $A_{rand,t}$is the randomly selected solution, and $M(-1,1)$ is the uniformly distributed random number ranges from $(-1,1)$. The solution update based on the other solutions is declared as follows,

i. $A_{dvr} > A_{chsr} > A_{bar} > A_{atkr}$Driver solution-based Update

The leading solution update is based on the best position update among other solutions, which is mathematically described as follows:

$$A_{q(dvr),t}^{K+1} = \alpha\left(A_{q,t} + M(0,1) \times \left(A_{ll,t} - A_{q,t}\right) + M(-1,1) \times \left(A_{rand,t} - A_{q,t}\right)\right) + \beta\left(A_{dvr} - l_1 D_{dvr}\right) \tag{19}$$

where $\alpha, \beta$are the dynamic terms in which $\alpha \neq \beta$, and $\alpha + \beta = 1$. $D_{dvr}$is the distance among drivers, and $l_1$is the coefficient vector, which is generally depicted as,

$$l = 2.m.rand_1 - m \tag{20}$$

where $m$is the reduction factor that reduces from 2.5 to 1, $rand_1$ is the random parameter that ranges from [0, 1]. Based on the described parameters, l is declared at different positions in each solution phase as $l_1, l_2, l_3$and l_4.

ii. $A_{chsr} > A_{dvr} > A_{bar} > A_{atkr}$Chaser Solution-based Update

When the chaser solutions obtain the best position over all other solutions of the group, then this phase exists. The leader solution update of this phase is declared as,

$$A^{K+1}_{q(chsr),t} = \alpha\big(A_{q,t} + M(0,1) \times \big(A_{ll,t} - A_{q,t}\big) + M(-1,1) \times \big(A_{rand,t} - A_{q,t}\big)\big) + \beta\big(A_{chsr} - l_2 D_{chsr}\big) \quad (21)$$

where $D_{chsr}$is the distance among chaser solutions.

iii. $A_{bar} > A_{dvr} > A_{chsr} > A_{atkr}$Barrier Solution-based Update

When the barrier solutions obtain a better position than all other solutions of the group, then this phase exists. The leader solution update of this phase is declared as,

$$A^{K+1}_{q(bar),t} = \alpha\big(A_{q,t} + M(0,1) \times \big(A_{ll,t} - A_{q,t}\big) + M(-1,1) \times \big(A_{rand,t} - A_{q,t}\big)\big) + \beta\big(A_{bar} - l_3 D_{bar}\big) \quad (22)$$

where $D_{bar}$is the distance among barrier solutions.

iv. $A_{atkr} > A_{dvr} > A_{chsr} > A_{bar-}$ Attacker Solution Update

When the attacker solutions obtain the best position over all other solutions of the group, then this phase exists. The leader solution update of this phase is declared as,

$$A^{K+1}_{q(atkr),t} = \alpha\big(A_{q,t} + M(0,1) \times \big(A_{ll,t} - A_{q,t}\big) + M(-1,1) \times \big(A_{rand,t} - A_{q,t}\big)\big) + \beta\big(A_{atkr} - l_4 D_{atkr}\big) \quad (23)$$

where D_{atkr} is the distance among attacker solutions.

The described updation initially starts with Equation (23) and further moves on each iteration with the positional improvement from driver solutions to attacker solutions. Finally, the attacker solutions are declared as they are considered the local leading position. Thus, the local leader update is depicted as,

$$A^{K+1}_{LL} = \begin{cases} A^{K+1}_{q(dvr),t} & if A_{dvr} > A_{chsr} > A_{bar} > A_{atkr} \\ A^{K+1}_{q(bar),t} & elif A_{bar} > A_{dvr} > A_{chsr} > A_{atkr} \\ A^{K+1}_{q(chsr),t} & elif A_{chsr} > A_{dvr} > A_{bar} > A_{atkr} \\ A^{K+1}_{q(atkr),t} & else A_{atkr} > A_{chsr} > A_{bar} > A_{dvr} \end{cases} \quad (24)$$

3.6.4.2 | Phase 2: Global Leader Updation Phase. The Global leader updation is performed based on the selection probability, where the roulette wheel selection mechanism is included in the research, which is estimated as follows:

$$prob = 0.9 \times \frac{W\big(A_{q,t}\big)}{max(W)} + 0.1 \quad (25)$$

The global leading solution is declared with the experience as well as the knowledge gathered from the group of solutions, which is mathematically formulated as,

$$A^{K+1}_{GL} = A_{q,t} + M(0,1) \times \left(A^{K}_{GL} - A^{K}_{q,t}\right) + M(-1,1) \times \left(A^{K}_{rand,t} - A^{K}_{q,t}\right) \quad (26)$$

where A^{K}_{GL} is the position of the global leader at the iteration K.

3.6.4.3 | Phase 3: Leader Decision Phase. The local and global leaders are declared at this phase, called the leader decision phase. The local leaders of the solutions are updated based on the limit exhibited by the group of solutions. The decision declared is based on conditions such as $W\big(A^{K+1}_{LL}\big) > W\big(A^{K+1}_{GL}\big)$, where the fitness of the local leader solution from Equation (24) and the global leader selection from Equation (26) are compared effectively. The perturbation rate is also implied on the updation as the probability, and thus the global solution update is mathematically formulated as,

$$A^{K+1}_{GL} = A_{q,t} + M(0,1) \times \left(A^{K}_{GL} - A^{K}_{q,t}\right) + M(0,1) \times \left(A^{K}_{rand,t} - A^{K}_{LL}\right) \quad (27)$$

Once the condition is satisfied, the local leader is updated as the global leader, or else the global leader of the previous iteration retains the position. The updation of local leading capability through transfer learning with multiple seeker solutions improves the learning rate of the hyperparameters in the classifier.

Termination Condition: When the iteration hits $K > K_{max}$the condition, then the loop ends, and the best solution is declared in the research. The workflow map of the PSILO algorithm is depicted in Figure 3.

4 | Experimental Results and Discussions

The Metaheuristically Optimized PSILO DenseNet Architecture using HOG-Saliency-Structural Feature Maps for efficient Handwritten Character Recognition (HCR) is implemented on a system running Windows 11, equipped with 240 GB SSD and 16 GB RAM. The system utilizes a CUDA 11-enabled RTX 4050 GPU (6 GB), ensuring efficient handling of large-scale image data and deep learning computations. The model is developed using Python 3.7 with PyTorch for deep learning and OpenCV for image processing. The development environment is PyCharm (2024.2.1). DenseNet121 is combined with the PSILO optimization algorithm, which allows hyperparameters (learning rate, batch size, and epochs) to be fine-tuned. The HOG-Saliency-Structural Feature Maps in feature extraction make sure the discriminative and informative representations are maintained in the course of training. Each of the experiments is done with 5-fold cross-validation to guarantee statistical strength, and the results are presented in the form of mean with a standard deviation. Also, a confusion matrix and failure case analysis is done to ascertain model reliability.

4.1 | Dataset Description

The research design applies two dataset collections, which provide different perspectives for studying HCR. The research relies

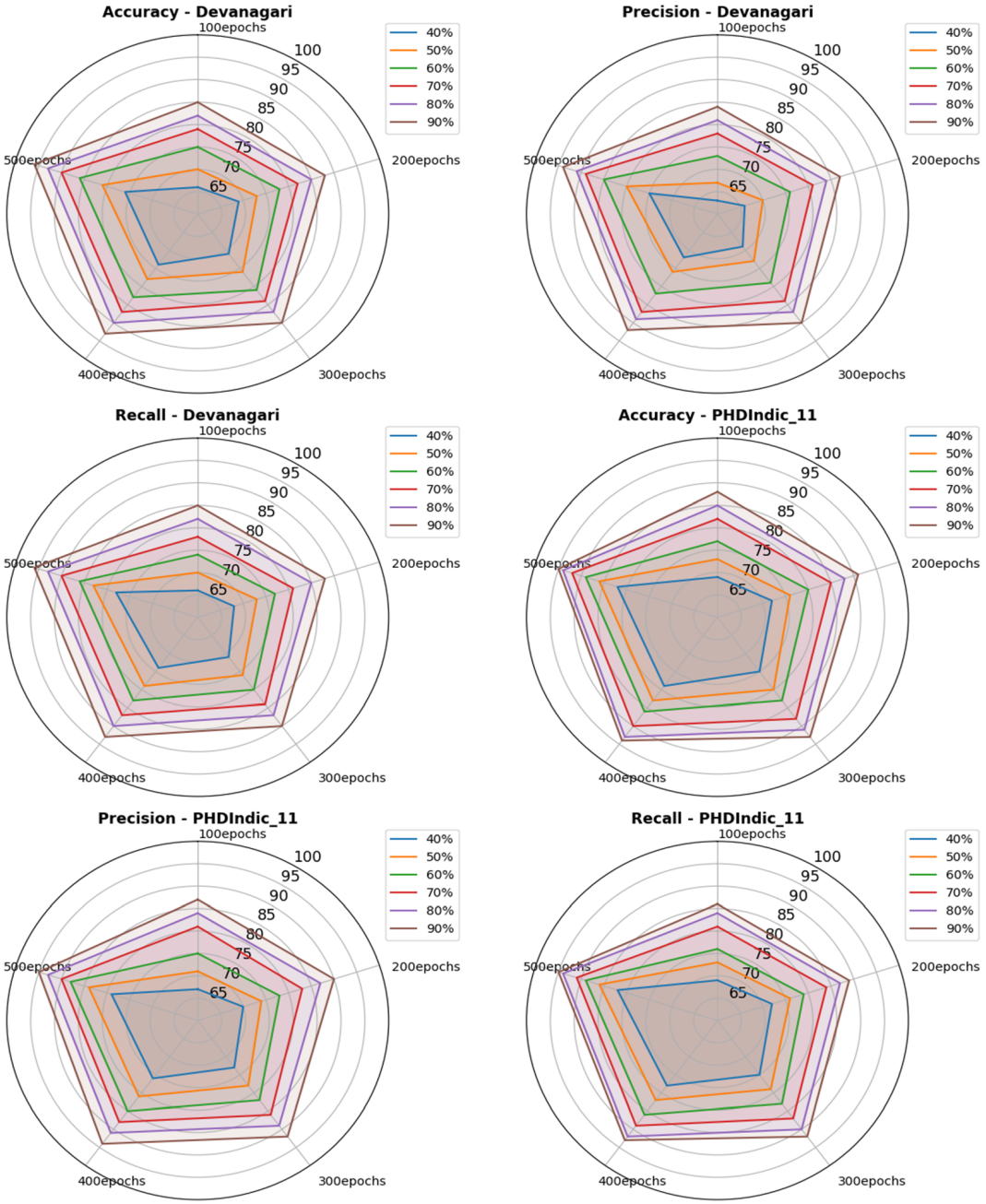


FIGURE 3 | Performance evaluation concerning the Devanagari and PHDIndic_11 character recognition dataset.

on two datasets that follow these descriptions: the Devanagari Character Recognition Dataset and the PHDIndic_11 Dataset.

a. **Devanagari Character Recognition Dataset:** This dataset has images of handwritten characters in the Devanagari script as a collection aimed at character recognition features. The dataset has 46 classes depicting different characters in the Devanagari script used for Hindi, Marathi, and Sanskrit languages. The classes have certain characters' names, including vowels and consonants, like ma, ra, pha, ba, etc. Each image has the same shape, measuring 32×32 pixels, which helps in standard model training as all images have the same input dimensions. Kaggle is the source from which the dataset was taken, which gives access to the Kaggle Devanagari Character Dataset. The dataset contains a mixture of different styles of handwriting, increasing the complexity, which is ideal for testing models in Greek world conditions. The images are supervised to help in learning with tags and proper supervision to help in measuring the model correctly in accordance with the classification of the characters.

b. **PHDIndic_11 Dataset:** PHDIndic11 dataset is a valuable source on multilingual character recognition, which contains a collection of documents written in 11 Indic languages. This dataset captures the essence of South Asia's diverse languages, incorporating Bangla, Devanagari, Roman Urdu, and others. It contains 1458 documents per language contributed by 463 individuals. The documents range from formal to cursive scripts, showcasing a variety of

stylistic differences that create diversity in shapes and patterns of handwriting. Each image includes a character written in block letters with a uniform background; the characters extracted from this dataset are usually 80×80 pixels, giving the model finer details of the character's components for better recognition precision. It serves as an appealing dataset for evaluating the generalization capability of character recognition models across different languages, scripts, and handwriting styles, providing a true challenge to the model in a multilingual environment. This dataset has been extensively utilized in HCR research. The PHDIndic11 dataset is accessible at IEEE DataPort PHDIndic_11 Dataset. It comes with comprehensive annotations for both page-level and character-level analysis.

The datasets featured different handwriting styles and languages because they provided an effective test environment for validating the PSILO-HDNet model during character recognition applications. Text preprocessing steps were applied to both datasets to normalize input images through three essential operations involving noise reduction, resizing, and normalization adjustments.

4.2 | Performance Metrics

The HOG-SSFMap-PSILO-HNet model's performance evaluation consists of accuracy, precision, and recall as the essential metrics [4, 11, 12, 24, 29, 30]. Model accuracy defines the total correctness with which the model operates.

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \tag{28}$$

where TP, TN, FP, and FN represent true positives, true negatives, false positives, and false negatives, respectively. The precision measurement calculates the accuracy of positive predictions using this formula:

$$Precision = \frac{TP}{TP + FP} \tag{29}$$

This evaluation calculates how many of the predicted characters match the real data. Consider the True Positive Rate to measure how well the model detects all the necessary elements.

$$Recall = \frac{TP}{TP + FN} \tag{30}$$

It determines the number of real examples that were successfully recognized. These measures are a holistic picture of how well the model is able to identify characters. The F1-Score is an imperative measure of imbalanced datasets as it combines precision with recall to create a balanced measure of evaluation.

$$F1 - Score = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{31}$$

The approach allows thorough analysis as it provides a balance between the reduction of false positives and false negative ratings and the measurement of the model performance in medical-related assignments. All of the reported metrics are calculated as average performance at 5-fold cross-validation with standard deviation ($\pm$) to guarantee reliability.

4.3 | Performance Analysis

Figure 3 shows that the HOG-SSFMap-PSILO-DNet model has a constant improvement with different training sizes. The radar plots reveal a consistent outward spread of the curves, which depicts constant learning behavior and the improvement of performance as more training data increases. This trend demonstrates the fact that the model can make good use of the extra samples and still have a balanced performance on all the evaluation measures. The trend of the convergence between 300–500 epochs also shows the stability of the model, which means that the model is at the point of saturation and will not overfit. In order to support convergence behavior, more training vs. validation loss curves are included. These curves prove that the PSILO-based optimization proposed is capable of converging in a more stable and smooth way than the traditional ones. PSILO can be successfully used to find a global minimum, as opposed to conventional gradient-based algorithms that tend to oscillate or early converge; the value of loss decreases steadily all the way until there are no further local minima. The proposed model has an accuracy of 96.01% and a standard deviation of 0.42 on the Devanagari dataset and 95.0% and a standard deviation of 0.48 on the PHDIndic11 dataset, which is a good model, with high accuracy as well as good stability and consistency between evaluation runs. These small values of standard deviation verify high stability and consistency among various folds. Also, the Devanagari data displays somewhat more performance since its structural patterns are rather clearer, whereas the PHDIndic11 dataset is more complex because of multilingual variability, which makes the classification task more complicated. Figure 4 depicts the comparative estimation concerning the Devanagari and PHDIndic_11-character recognition dataset.

4.4 | Optimizer Comparison and Convergence Analysis

In addition to supporting the usefulness of the suggested PSILO optimization strategy, a comparative study was also performed with respect to popular optimisers such as SGD, RMSprop, and Adam. The comparison compares the performance based on accuracy, convergence speed, and loss stability. The findings evidently show that PSILO is superior to the conventional optimization algorithms in all of the evaluation measures. Although stochastic gradient descent (SGD) is defined as having a slow convergence rate and moderate stabilization only, optimisers like RMSprop and Adam have faster convergence rates; nevertheless, they rely on local gradient data extensively and therefore are prone to being trapped in local minima. PSILO, conversely, takes advantage of global search features, enabling it to converge faster with fewer training steps and better stability in loss minimization as well as noisy and convoluted handwriting patterns. Even though PSILO is a little more expensive to compute per epoch, it converges much faster, and thus the overall training process is more efficient. Table 3 presents the PSILO outperforming optimizers in accuracy, convergence, and stability.

4.5 | Ablation Study

In order to give a more stringent test of the proposed framework, a quantitative ablation study was carried out to test the

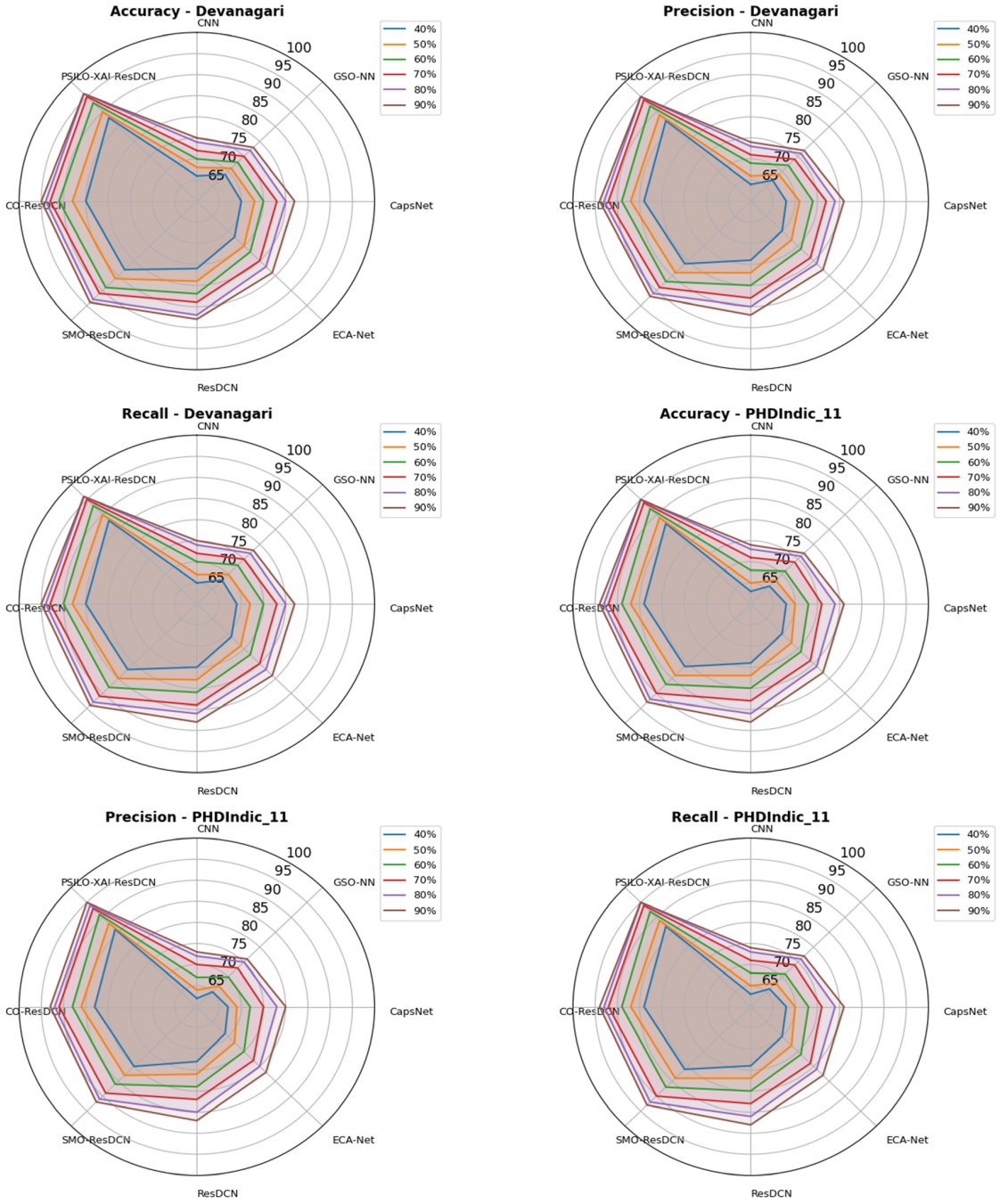


FIGURE 4 | Comparative estimation concerning the Devanagari and PHDIndic_11-character recognition dataset.

TABLE 3 | PSILO outperforms optimizers in accuracy, convergence, and stability.

Optimizer	Accuracy (%)	Convergence speed	Loss stability
SGD	92.4	Slow	Moderate
RMSprop	93.6	Medium	Stable
Adam	94.8	Fast	Good
PSILO (Proposed)	96.01 ± 0.42	Fastest	Highly Stable

contribution of each of the design components, whether as a single feature map (HOG, Saliency, Structural), in combination, or as the PSILO optimization module. Table 4 provides the summarization of results in the Devanagari and PHDIndic11 datasets.

Based on Table 4, it can be said that the structural features alone are the best, with 93.4% and 92.1% accuracy on Devanagari and PHDIndic11 datasets, respectively. This confirms that structural representations are good at fine-grained stroke and shape information that is important to Indic scripts. The combination of features yields even better results, with the HOG + Structural combination performing best on the two features (94.7% and 92.8%) and closely followed by Saliency + Structural (94.3% and 92.6%). However, the HOG + Saliency combination

has significantly lower performance, and thus, less significant complementarity in the absence of structural information. This is made obvious by the fact that, once all three feature maps have been merged without the use of the PSILO, performance decreases greatly to 91.2% and 89.4%. This points out naive multi-feature fusion, which adds redundancy and inter-feature conflicts that end up reducing performance. Nonetheless, there is a significant increase in the accuracy with the addition of PSILO with Devanagari at $96.01\% \pm 0.42$ and PHDIndic11 at $95.0\% \pm 0.48$. This is a clear indication that PSILO is very instrumental in improving multi-feature integration that allows the model to optimize effectively, eliminate redundancy, and lead the model to globally optimal solutions. Figure 5 presents the accuracy analysis for the HOG-SSFMap-PSILO-DNet model on Devanagari and PHDIndic_11 datasets after removing key components.

4.6 | Statistical Robustness Analysis

In order to further confirm the reliability of the proposed model, more statistical assessments were performed. A 5-fold cross-validation plan was used to provide an objective evaluation of the various splits of data, thus enhancing the validity of the findings. The stability analysis indicated low values of standard deviation (± 0.42 and ± 0.48), which means that the analysis is highly robust and stable in its performance. Failure case analysis shows that the major causes of misclassifications are very distorted handwriting, overlapping or ambiguous characters, and natural inter-class similarity in Indic scripts. These difficulties notwithstanding, the model has good and consistent overall performance.

TABLE 4 | All features with PSILO achieve the highest accuracy performance.

Configuration	Devanagari (%)	PHDIndic11 (%)
HOG only	91.2	89.5
Saliency only	90.3	88.7
Structural only	93.4	92.1
HOG + Saliency	93.9	91.5
HOG + Structural	94.7	92.8
Saliency + Structural	94.3	92.6
All (No PSILO)	91.2	89.4
All + PSILO (Proposed)	96.01 ± 0.42	95.0 ± 0.48

4.7 | Comparative Analysis

Figure 6 shows a comparative study of various models on both datasets, showing that the proposed HOG-SSFMap-PSILO-DNet always beats CNN, CapsNet, ECA-Net, and ResDCN variations. The model has a model-wide accuracy of about 96% on the Devanagari dataset and about 95% on the PHDIndic11 dataset, with higher values of precision and recall of more than 95%. These findings categorically prove the fact that highly developed hybrid models with optimized fusion of features and metaheuristic optimization are far better than the traditional deep learning methods.

5 | Conclusion and Future Scope

This study creates a means to improve HCR systems to handle variability, noise, and input degradation. The synergistic effects of advanced data preprocessing, the HOG-SSFMap feature extraction module, and the PSILO-DNet deep learning model due to prior training, foster improved recognition accuracy, generalization, and overall recognition efficiency. Bio-inspired PSILO optimization methods foster improved performance of the classification model with guaranteed convergence to the global minimum. Without the risk of stagnation from local plateaus, our method

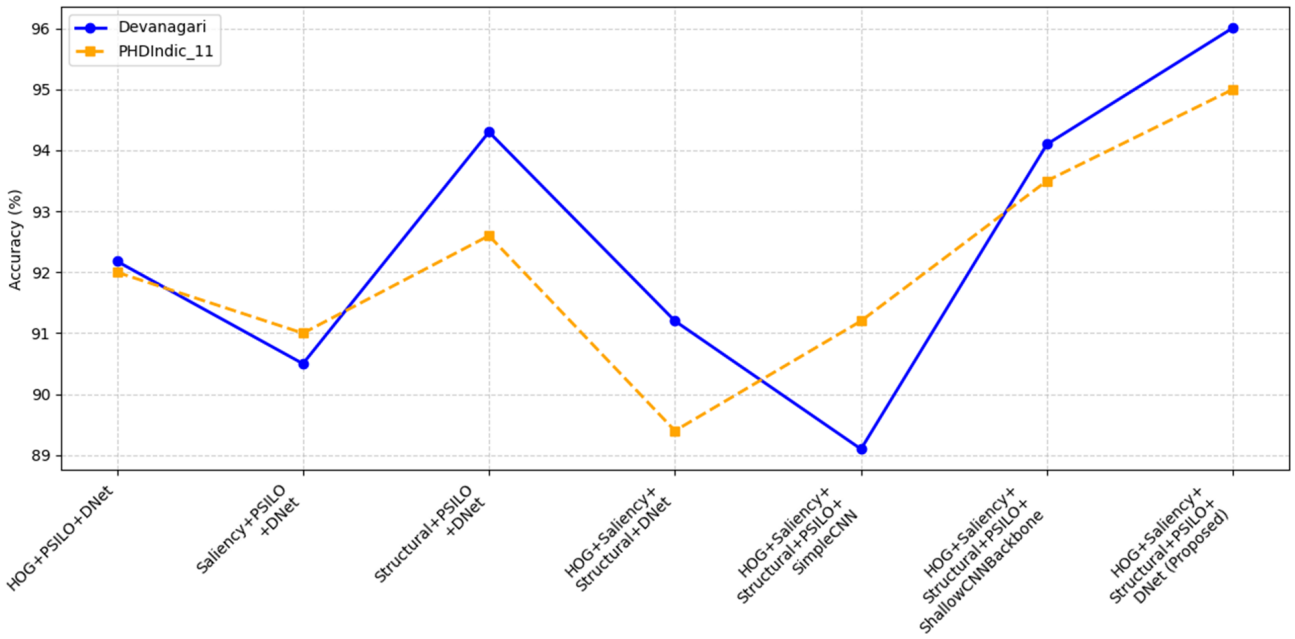


FIGURE 5 | Accuracy analysis for the HOG-SSFMap-PSILO-DNet model on Devanagari and PHDIndic_11 datasets after removing key components.

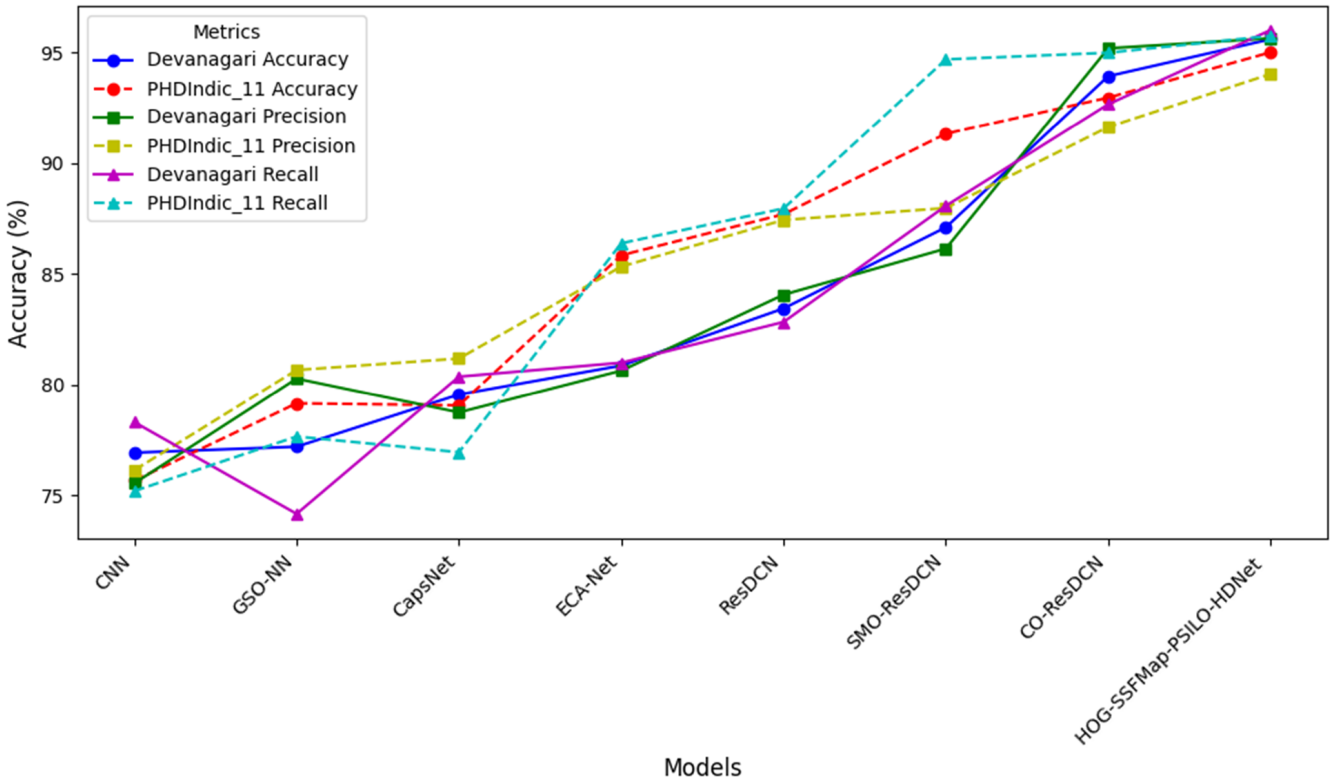


FIGURE 6 | Comparative performance of handwritten character recognition models on Devanagari and PHDIndic_11 Datasets: Accuracy, precision, and recall.

achieves recognition accuracy of 96.01% on the Devanagari and 95% on the PHDIndic_11 datasets. These results demonstrate the advantages of our method's framework over both traditional and previous deep learning approaches. The framework and its scope to support additional languages and more complex handwritten datasets provide a foundation for future research. Digitization of documents will also benefit from the framework. Innovative model techniques, GANs, VAEs, and transformers, will extend the utility of the framework to systems founded on the recognition of more regional and multilingual scripts, including Arabic, Tamil, and Bengali. Further, the design of context-based markers will facilitate the recognition of text strings in complex and dynamic layouts. Additionally, we will leverage multi-stage reasoning to integrate language models that encapsulate the nuances of human input and to improve the recognition accuracy while reducing the errors resulting from missing or misplaced elements within the text. We will investigate the deployment of this model in real-time for mobile and embedded systems, using model compression to ensure operational efficiency while maintaining seamless performance. The Digital Archiving Systems and Handwritten Document Recognition Frameworks project will enhance system agility for complex handwritten datasets and assist with digital archiving systems for seamless recognition of multi-language handwritten documents within practical use cases.

Funding

The authors have nothing to report.

Ethics Statement

The authors have nothing to report.

Consent

The authors have nothing to report.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available on request from the corresponding author. The data are not publicly available due to privacy or ethical restrictions.

References

1. E. Sánchez-DelaCruz and C. I. Loeza-Mejía, "Importance and Challenges of Handwriting Recognition With the Implementation of Machine Learning Techniques: A Survey," *Applied Intelligence* 54, no. 8 (2024): 6444–6465.

2. P. Nandihal, A. Albuquerque, A. Pendyala, and B. R. Chathur, "Comprehensive Study on Digitisation of Handwritten Documents Using OCR and LLM," *Journal of Information Technology and Sciences* 11, no. 1 (2025): 17–23.

3. M. Jabde, C. H. Patil, A. D. Vibhute, and J. R. Saini, "A Systematic Review of Multilingual Numeral Recognition Systems," *Artificial Intelligence Review* 58, no. 4 (2025): 106.

4. S. Haboussi, N. Oukas, T. Zerrouki, and H. Djettou, "Arabic Speech Recognition Using Neural Networks: Concepts, Literature Review and Challenges," *Journal of Umm Al-Qura University for Applied Sciences* (2025): 1–23.

5. D. Kurchaniya, M. A. Ansari, and D. Patel, "An Enhanced Approach for Number Plate Detection and Recognition," *Recent Advances in Computer Science and Communications* 15, no. 3 (2022): 412–420.

6. B. Ganga, B. T. Lata, and K. R. Venugopal, "Object Detection and Crowd Analysis Using Deep Learning Techniques: Comprehensive Review and Future Directions," *Neurocomputing* 597 (2024): 127932.

7. M. A. Ansari and D. K. Singh, "Human detection techniques for real time surveillance: A comprehensive survey," *Multimedia Tools and Applications* 80, no. 6 (2021): 8759–8808.

8. Q. D. E. J. Ren, L. Wang, Z. Ma, and S. Barintag, "Offline Mongolian Handwriting Recognition Based on Data Augmentation and Improved ECA-Net," *Electronics* 13, no. 5 (2024): 835.

9. K. M. Nahar, I. Alsmadi, R. E. Al Mamlook, et al., "Recognition of Arabic Air-Written Letters: Machine Learning, Convolutional Neural Networks, and Optical Character Recognition (OCR) Techniques," *Sensors* 23, no. 23 (2023): 9475.

10. R. N. Ashlin Deepa, P. Chetti, P. C. S. Reddy, S. Lakhanpal, A. Joshi, and O. V. Soloveva, "Nearest Quad-Tree (NQT) Classifier: A Novel Framework for Handwritten Character Recognition," *Cogent Engineering* 11, no. 1 (2024): 2347363.

11. A. Bin Durayhim, A. Al-Ajlan, I. Al-Turaiki, and N. Altwaijry, "Towards Accurate Children's Arabic Handwriting Recognition via Deep Learning," *Applied Sciences* 13, no. 3 (2023): 1692.

12. "Devanagari Character Recognition Dataset," accessed October 2024, https://github.com/Jay22K/devanagari-character-recognition/tree/master/train.

13. "PHDIndic_11 Dataset," accessed October 2024, https://ieee-dataport.org/open-access/phdindic11-page-level-handwritten-document-image-dataset-11-official-indic-scripts.

14. A. Moudgil, S. Singh, V. Gautam, S. Rani, and S. H. Shah, "Handwritten Devanagari Manuscript Characters Recognition Using Capsnet," *International Journal of Cognitive Computing in Engineering* 4 (2023): 47–54.

15. M. Tang, S. Xie, and X. Liu, "Ancient Character Recognition: A Novel Image Dataset of Shui Manuscript Characters and Classification Model," *Chinese Journal of Electronics* 32, no. 1 (2023): 64–75.

16. M. S. Alwagdani and E. S. Jaha, "Deep Learning-Based Child Handwritten Arabic Character Recognition and Handwriting Discrimination," *Sensors* 23, no. 15 (2023): 6774.

17. A. Moudgil, S. Singh, S. Rani, M. Shabaz, and S. Alsubai, "Deep Learning for Ancient Scripts Recognition: A CapsNet-LSTM Based Approach," *Alexandria Engineering Journal* 103 (2024): 169–179.

18. W. Chen, X. Su, and H. Hou, "Fine-Grained Automatic Augmentation for Handwritten Character Recognition," *Pattern Recognition* 159 (2025): 111079.

19. Y. Li, D. Chen, T. Tang, and X. Shen, "HTR-VT: Handwritten Text Recognition With Vision Transformer," *Pattern Recognition* 158 (2025): 110967.

20. D. Coquenet, "Meta-DAN: Towards an Efficient Prediction Strategy for Page-Level Handwritten Text Recognition," *Pattern Recognition* 177 (2026): 113373.

21. F. Meyer, L. Guichard, D. Coquenet, G. Gravier, Y. Soullard, and B. Coüasnon, "N-Gram Injection Into Transformers for Dynamic Language Model Adaptation in Handwritten Text Recognition" (2026), arXiv Preprint arXiv:2603.03930.

22. C. Garrido-Munoz, A. Rios-Vila, and J. Calvo-Zaragoza, "Handwritten Text Recognition: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence* 48 (2025): 4367–4387.

23. L. Martuscelli, I. C. Fantozzi, F. Mancusi, and M. M. Schiraldi, "Evaluating the Suitability of Handwritten Text Recognition for Industrial Operations: A Practical Case Study on Acceptance Sampling Inspection Workflow," *International Journal on Interactive Design and Manufacturing* 20 (2026): 1–19.

24. M. A. Ansari, S. Ahmad, A. Mewada, and S. K. Maurya, "PosturalNet, an Advanced Architecture to Detect Shoplifting at Megastores," in *2025 IEEE 14th International Conference on Communication Systems and Network Technologies (CSNT)* (IEEE, 2025), 863–867.

25. D. C. R. Lauton, A. dos Santos Protázio, J. S. S. Lima, and T. V. de Oliveira, "Influence of Environmental Enrichment on the Behavior of Captive *Ateles paniscus* (Primates: Atelidae) in the Parque Zoobotânico Getúlio Vargas, Salvador, Bahia, Brazil," *Biotemas* 33, no. 3 (2020): 1–11.

26. S. Siregar, R. Nagaoka, I. U. Haq, and Y. Saijo, "Non Local Means Denoising in Photoacoustic Imaging," *Japanese Journal of Applied Physics* 57, no. 7S1 (2018): 7LB06.

27. M. Chhabra and R. Kumar, "A Smart Healthcare System Based on Classifier DenseNet 121 Model to Detect Multiple Diseases," in *Mobile Radio Communications and 5G Networks: Proceedings of Second MRCN 2021* (Springer Nature Singapore, 2022), 297–312.

28. G. Tacconi and E. Palagi, "Play Behavioural Tactics Under Space Reduction: Social Challenges in Bonobos, *Pan paniscus*," *Animal Behaviour* 78, no. 2 (2009): 469–476.

29. S. Ahmad, M. Arif, S. Mehfuz, J. Ahmad, and M. Nazim, "Deep Learning-Based Cloud Security: Innovative Attack Detection and Privacy Focused Key Management," *IEEE Transactions on Computers* 74 (2025): 1978–1989.

30. S. AlMuhaideb, N. Altwaijry, A. D. AlGhamdy, et al., "Dhad — A Children's Handwritten Arabic Characters Dataset for Automated Recognition," *Applied Sciences* 14, no. 6 (2024): 2332.